EQUADIS

Supplier & Retailer Partnership Portal

From one marketplace to many - design document and proof of concept

Senior Software Engineer technical test - Part A (design) and Part B (proof of concept)

Prepared by: Mostafa Haidar Ahmad

Date: November 3, 2026

Repository: this document, the proof of concept (Java 21, Spring Boot 3.5, PostgreSQL 16), README.md, EDGE_CASES_AND_DECISIONS.md

How to read this document

1 the problem, its numbers and my assumptions · **2** the solution, and how it fits Equadis' platform · **3** the data model · **4** each need, R1 to R10, including what happens when things go wrong · **5** decisions and alternatives · **6** the path from today and what I would not build yet · **7** the proof of concept.

Every edge case handled is listed in `EDGE_CASES_AND_DECISIONS.md`.

Contents

Executive summary 2
1. Understanding the problem 3
1.1 Context 3
1.2 The numbers that matter 3
1.3 Assumptions (and what would change my decision) 4
2. Solution overview 4
2.1 Main parts and layers 4
2.2 Request pipeline (middleware) and error handling 5
2.3 How this fits Equadis' platform 5
3. Data model 6
4. How the solution meets each need 7
R1 - Several marketplaces, never mixed 7
R2 - Growth 7
R3 - Offers from supplier systems (file or API) 7
R4 - Fast catalogue (p95 < 200 ms) 8
R5 - Being informed of new offers 8
R6 - End of a partnership 9
R7 - Security and compliance 10
R8 - Availability 99.9% and zero-downtime releases 10
R9 - Daily extract · R10 - Branding 10
5. Main technical decisions and alternatives 11
Risks and mitigations 11
6. From today's application to the target 11
6.1 Migration path (each step shippable) 11
6.2 What I would deliberately not build yet 12
7. Proof of concept 12
Use of AI assistants 13

Executive summary

The portal serves one marketplace today. In 18 months it must serve about 20 marketplaces that never see each other's data, grow about 100x (5,000 suppliers, 300,000 retailers, 3 million offers), accept up to 100,000 offers at a time from supplier systems, answer the catalogue in under 200 ms, and inform retailers of new offers - including on Black Friday, when 500,000 offers are published in one hour. My answer, in seven decisions:

1. **One application, a modular monolith** in three layers per module (controller, service, repository), enforced by an automated architecture test. The team and the load do not justify microservices.
2. **Multi-tenancy in layers:** `marketplace_id` on every row, enforced by PostgreSQL **Row-Level Security** (a forgotten WHERE clause still cannot leak data), and **cells** - the same image with its own database - for customers who require separate storage.
3. **Rules that concurrency can break live in the database:** unique indexes, an exclusion constraint against overlapping partnerships, composite foreign keys. Java gives friendly messages; the database gives the guarantee.
4. **One import pipeline for every format.** Inbound connectors (CSV file, JSON API) turn offers into one canonical form; validation, dry run, error report, duplicate protection and crash recovery are shared. Each 1,000-line chunk is one transaction with a checkpoint.
5. **Notifications through a transactional outbox, coalesced into digests** computed at send time: Black Friday produces about 300,000 digests instead of about 450 million messages.
6. **Status derived, not stored:** partnership status comes from dates and "today" (Europe/Paris); catalogue visibility is computed at read time, so ending a partnership removes access instantly (R6).
7. **Boring, proven technology first:** PostgreSQL does the queueing, search and isolation at this scale. Kafka, Elasticsearch and sharding are written down as "when we reach X", not built.

The proof of concept implements R1, R3 and R5 plus the rules around them (R6, R7 audit, BR-03 to BR-11). It starts with `docker compose up` and is covered by **154 automated tests** (88 unit, 66 integration against a real PostgreSQL).

1. Understanding the problem

1.1 Context

Equadis connects brands and retailers around product data (GS1-certified data pool, GDSN, 950+ connectors). This portal applies the same idea to **commercial offers**: brands (suppliers) and stores (retailers) of a **marketplace** work together through time-bound partnerships; suppliers publish discounts; partnered retailers browse and subscribe to them. **A marketplace** (assumption) is one Equadis customer - a retail cooperative or a buying alliance - with its own private network, branding and reports; the same real brand in two marketplaces has two separate accounts.

1.2 The numbers that matter

Figure	Derivation	Design consequence
~9,000 offers visible per retailer	15 partners x ~600 offers	Keyset pagination, filters and search; never "load all".
~900 retailers per supplier	300,000 x 15 / 5,000	Publishing concerns ~900 retailers: fan-out in one set-based statement.
~450 million (offer, retailer) pairs on Black Friday	500,000 offers x ~900 retailers	Per-offer messages are impossible: coalesced digests.
~140 offers/s sustained	500,000 / 3,600 s	JDBC batching; one outbox event per chunk, not per offer.
100,000 offers per import	R3	Streaming, chunked transactions, checkpoint.
p95 < 200 ms, 99.9%	R4, R8 (~43 min down/month)	2+ stateless instances, read replica, background work isolated.
3M offers ≈ 1-2 GB with indexes	~300 B per row + indexes	Fits one PostgreSQL instance: no sharding.

1.3 Assumptions (and what would change my decision)

Assumption	If wrong, I would...
"Stored separately" (R1) means a dedicated database with its own backups and keys, same region.	Deploy that customer's cell in the required region or account; the design already supports it.
Partnership dates are inclusive; a renewal starts the day after the current end (today if expired).	Change one comparison and the range bound; or extend the same row instead.
Offer references are case-insensitive and trimmed.	Remove `lower()` from the unique index and the comparison.
A file adds or updates offers; offers missing from it are not deleted (delta, not snapshot).	Add an explicit, opt-in "full replace" mode.
Only DRAFT offers change through an import; a different line for a PUBLISHED offer is a line error (BR-08).	Create a new version of the offer, linked to the old one, and close the old one.
Only the marketplace admin creates and renews partnerships; the admin or either partner can terminate.	Add a request/approve step between supplier and retailer.
Terminating a partnership also terminates its renewal waiting to start (R6).	Keep the renewal and say so in the termination response.
Subscriptions stay ACTIVE when their offer is closed; the offer leaves the catalogue. **Open question for product.**	Cancel them in the same transaction as the closure, as for termination.
Expiry (not termination) does not cancel subscriptions; BR-10 already hides the offers.	Cancel on expiry with a daily job. **Open question for product.**
An offer published with a future start date is announced only from the day it is visible (BR-10).	A daily job announcing offers that start today.

2. Solution overview

One codebase, one Docker image: a Spring Boot modular monolith deployed in **cells**. A cell is a complete copy of the platform (API and worker instances, PostgreSQL with a read replica, object storage). The shared cell hosts most marketplaces, isolated by Row-Level Security; a marketplace that requires separate storage gets its own cell. The edge routes each request to its cell by sub-domain.

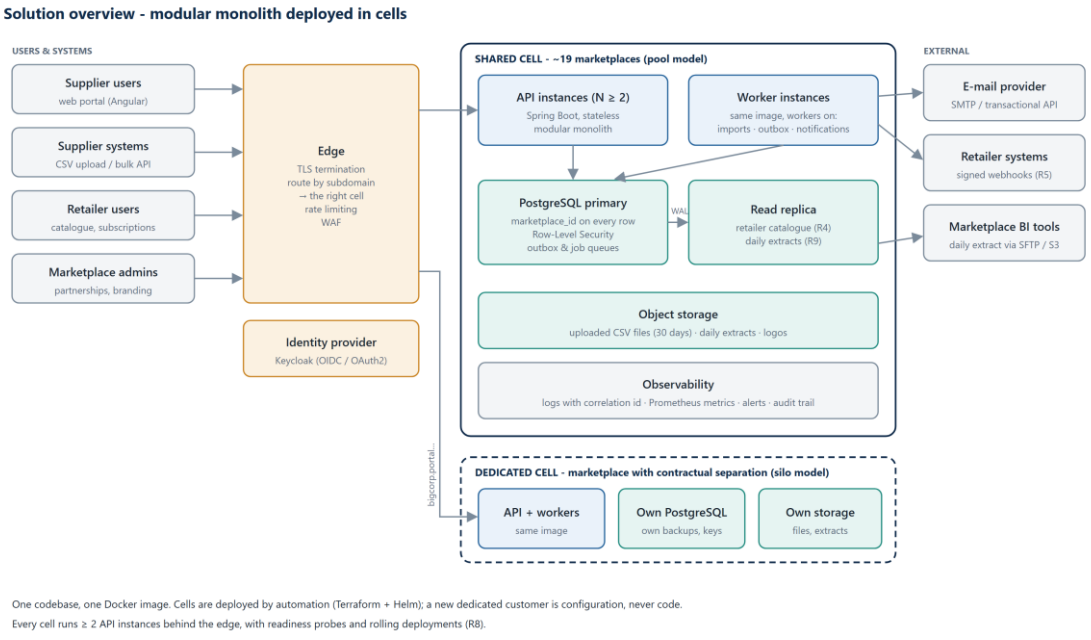


Figure 1 - Solution overview

2.1 Main parts and layers

Part	Responsibility
API instances	Stateless REST API for the Angular portal and for supplier/retailer systems.
Worker instances	The same image with background work enabled (imports, outbox, notifications, extracts), scaled separately so imports never slow the catalogue.
PostgreSQL primary + read replica	Source of truth and work queues (`FOR UPDATE SKIP LOCKED`); the replica serves the catalogue and reports.
Object storage, edge, identity	Uploaded files and extracts; TLS, routing, rate limiting, WAF; Keycloak (OIDC).

Inside, each business module (partnership, offer, catalogue, subscription, importing, outbox, notification, audit, directory) has the same three layers: **controllers** translate HTTP, **services** hold use cases, rules, authorization and transactions, **repositories** are the only classes with SQL. Modules call each other only through public services. Rules live in domain objects (`Partnership`, `Offer`, `OfferTerms`) unit-tested without Spring. Bulk paths use plain JDBC; single-entity use cases use JPA.

2.2 Request pipeline (middleware) and error handling

Every call goes through: **tracing** (correlation id in every log line and in the response) → **size limit** (1 MB; imports 50 MB, counted while reading) → **identity** (caller and marketplace) → controller → service, and a **global exception handler** that turns every failure into RFC 9457 Problem Details with a stable `code`, the business rule (`BR-08`), every violated field and the correlation id. 4xx are logged once without stack trace; 5xx with it, and the caller gets a generic message - internals never leak.

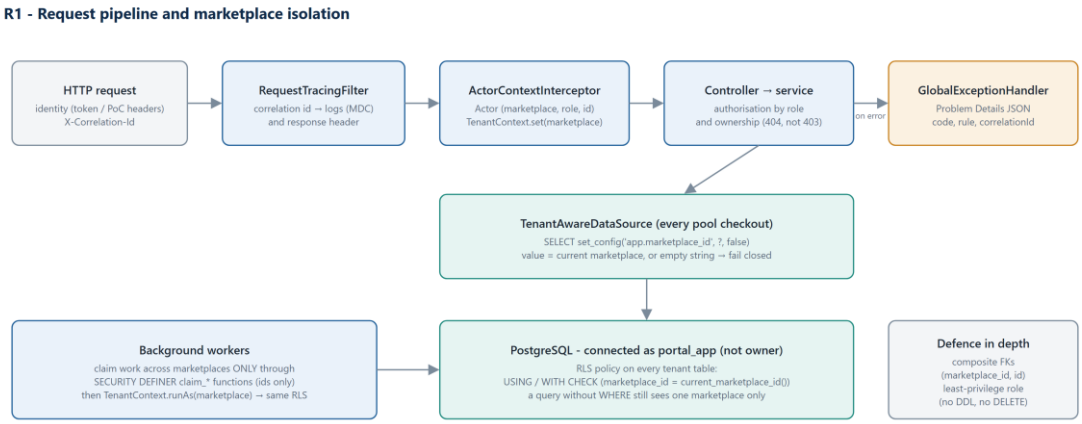


Figure 2 - Request pipeline and marketplace isolation

2.3 How this fits Equadis' platform

Equadis' product-data flow is: collect through connectors, standardise into one model, validate before publication, distribute adapted to each recipient, acknowledge, and monitor quality. The portal follows the same flow for offers, so it can share concepts, people and, later, infrastructure with the rest of the platform:

Equadis concept	In this design	Status
Connectors (GDSN, Excel matrices, APIs)	`OfferSourceAdapter`: CSV file and JSON API turned into one canonical form. A new format is one class.	Built
Validation engine "before publication"	One rule set (`OfferTerms`) for every channel, and a **dry run** that says what an import would do without changing anything.	Built
Data quality dashboards	Problems grouped by code per import ("38 x DATE_INVALID") and Prometheus metrics.	Built
Distribution adapted to each recipient	`NotificationChannel`: e-mail digest, signed webhook; marketplaces or GDSN would be further channels.	Built

Equadis concept	In this design	Status
Technical acknowledgment	Only a 2xx counts as delivered, with a delivery id against duplicates.	Built
Requirements per market	A rule set per marketplace (allowed currencies, maximum discount, mandatory fields), stored as configuration and applied by the same validation engine.	Designed

3. Data model

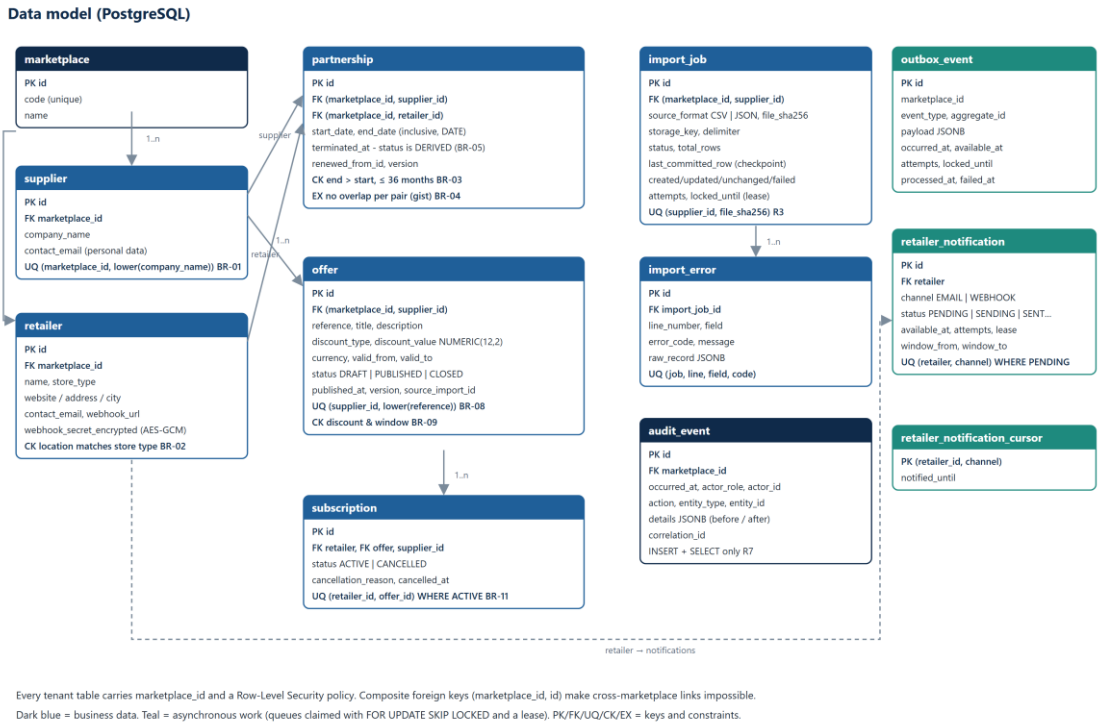


Figure 3 - Data model (PostgreSQL)

- **Every tenant table has `marketplace_id`** and a Row-Level Security policy; composite foreign keys `(marketplace_id, id)` make cross-marketplace links impossible.
- **Two kinds of time:** calendar dates (partnerships, validity) are `DATE` in Europe/Paris; moments are `TIMESTAMPTZ` in UTC. "Today" comes from one injected `Clock` and is passed to SQL - never `CURRENT_DATE`.
- **Money** is `NUMERIC(12,2)` / `BigDecimal`; free text is `VARCHAR(n)`; constraint names are explicit and translated into business errors, so a race that slips past Java still gets a 409, not a 500.
- **Supporting tables:** `import_job` / `import_error` (R3), `outbox_event`, `retailer_notification` + cursor (R5), `audit_event` (R7, append-only).

Rule	In Java (friendly message)	In the database (guarantee)
BR-01 supplier name unique per marketplace	-	unique index on lower(company_name)
BR-02 location by store type · BR-09 discount	`OfferTerms` (API and imports)	check constraints
BR-03 end > start, ≤ 36 months	`Partnership.validatePeriod`	check constraints (same arithmetic)
BR-04 no overlapping partnerships	pre-check query	**exclusion constraint** (gist, date range)
BR-05 status · BR-06 renewal	`Partnership.statusOn(today)`, `renew`	nothing stored, nothing to go stale
BR-07 termination cancels subscriptions	same transaction	row locks FOR UPDATE / FOR SHARE
BR-08 lifecycle, reference unique	`Offer` state machine	unique index; guarded batch UPDATE

Rule	In Java (friendly message)	In the database (guarantee)
BR-10 visibility · BR-11 one subscription	catalogue / digest queries; idempotent subscribe	joins on ACTIVE partnership; partial unique index

4. How the solution meets each need

R1 - Several marketplaces, never mixed

Shared cell. The application connects with a role that does **not** own the tables, so PostgreSQL applies Row-Level Security to every query (`marketplace_id = current_marketplace_id()`). The marketplace comes from the verified identity and is written to the connection on every checkout; with none, the database returns nothing (**fail closed**). Background workers find work through three reviewed `SECURITY DEFINER` functions that return ids only, then switch to that marketplace. **Dedicated cell:** the contractual customer gets the same image with its own database, storage, backups and keys, created by automation (Terraform, Helm); schema and code are identical everywhere.

> **When things go wrong**
> A forgotten WHERE clause → RLS still filters (test `databaseHidesOtherMarketplaces_evenWithoutWhereClause`). No identity → 401; no marketplace → empty results.
> Guessing another marketplace's id → 404, existence not revealed (UUIDs, not sequences).

Moving a marketplace to its own cell. Because ids are UUIDs and every row carries `marketplace_id`, a move is an export/import filtered by marketplace - no id clashes. Cut-over: rehearse on a copy, freeze that marketplace's writes for a few minutes (workers drain), copy and compare row counts, switch its sub-domain to the new cell, verify, then delete its rows from the shared cell. **Migrations across N cells:** the same Flyway scripts run at start-up in every cell, canary cell first; expand/contract keeps old and new releases compatible during the rollout.

R2 - Growth

100x growth fits one PostgreSQL per cell. What scales: stateless API instances, separate worker instances, a read replica, set-based SQL on heavy paths. Peak load stays around 1,000-2,000 short statements per second: if 10% of retailers open 20 catalogue pages in the busiest hour, that is ~170 requests/s x ~3 statements; sending ~170 notifications/s x ~4 statements; plus imports in batches of 1,000 and fan-out, which add little. Pools stay small per instance and PgBouncer joins when instances multiply (the marketplace setting then moves inside each transaction). Thresholds for bigger changes are in section 6.2.

R3 - Offers from supplier systems (file or API)

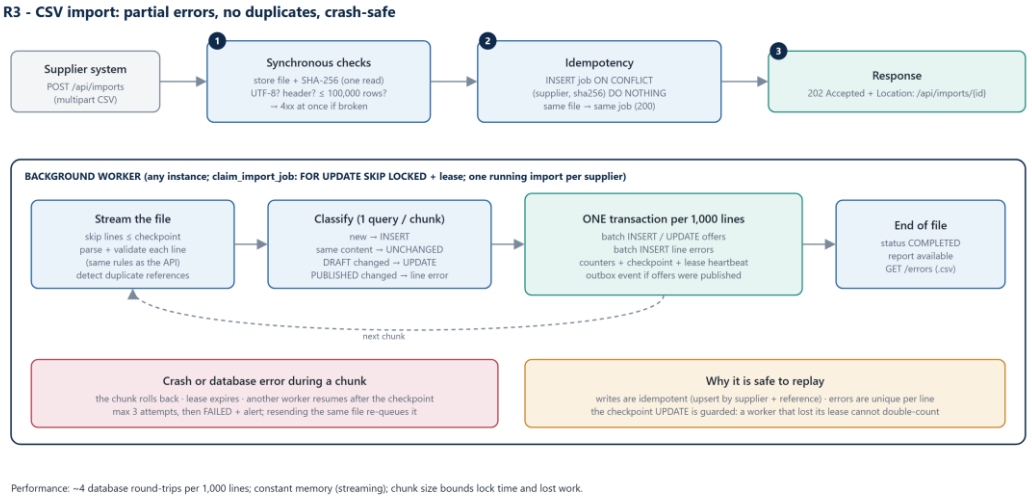


Figure 4 - Import pipeline

Intake (synchronous). An inbound connector turns the CSV file or the JSON request `{"offers": [...]}` into the canonical CSV while it streams, computing a SHA-256 on the way. The structure is checked at once (strict UTF-8, separator, columns, ≤ 100,000 offers, ≤ 50 MB): a broken file gets a 4xx, never 100,000 identical line errors. The job is inserted with `ON CONFLICT (supplier_id, file_sha256) DO NOTHING`: **the same offers sent twice return the first job**. A **dry run** (`/api/imports/dry-run`) runs the same checks and decisions and returns what would happen, without storing anything.

Processing (asynchronous). A worker claims the job (`SKIP LOCKED`, lease, one running import per supplier) and writes 1,000 offers per transaction: one query loads the chunk's existing offers, then batch INSERT/UPDATE, line errors, counters and checkpoint, one audit row and - if offers were published - one outbox event. Each offer is CREATED, UPDATED, UNCHANGED or FAILED; every problem is reported with its position (CSV line or JSON item), field name as the supplier sent it, stable code and message, grouped by code, as JSON or CSV (CSV-injection safe).

> **When things go wrong**
>
> Bad lines → only they fail, with every problem at once; hostile values (NUL characters, huge numbers, impossible dates, 100 KB cells) stay line errors.
>
> Same reference later → updated if DRAFT, unchanged if identical, line error if PUBLISHED and different (BR-08); the unique index forbids duplicates.
>
> Crash mid-file → the chunk rolls back, the lease expires, another worker resumes after the checkpoint; after 3 attempts the job is FAILED with a support reference, and re-sending re-queues it.

R4 - Fast catalogue (p95 < 200 ms)

One query starts from the retailer's ~15 live partnerships; for each one, a `LATERAL` sub-query reads that supplier's published offers newest-first from a **partial index** `(supplier_id, published_at DESC, id DESC) WHERE status = PUBLISHED` and **stops after one page**. The short lists are merged and cut to one page, so the work is bounded by partners x page size, however large a catalogue grows. **Keyset pagination** makes page 200 cost the same as page 1; page size and search text are bounded; text search uses `pg_trgm` in production. In production the catalogue runs in **two steps**: the retailer's active partner suppliers are read from the **primary** (a few rows, always up to date), then the offers of exactly those suppliers from the **replica**. So a termination applies at once even if the replica is a few seconds behind (R6); the PoC runs both steps as one query on the primary. **No cache at first.** Caching each retailer's pages and clearing them on termination looks simple but is fragile: expiry at midnight has no event (BR-05), a page read just before the termination commits can be written back after the clear, and every instance must be cleared. If measurements ever require a cache, it holds the **content, never the permission**: each supplier's published offers, shared by its ~900 retailers and cleared when that supplier publishes or closes; the partner list is still read live on every request, so R6 never depends on the cache. Proof before go-live: `EXPLAIN ANALYZE` on production-like data and a k6 load test in CI.

> **When things go wrong**
>
> Replica down → the health check removes it and offer reads go to the primary, which can carry them for a while; replica lagging → new offers appear a few seconds late, but nothing leaks, because the partner list always comes from the primary.
>
> Database under heavy load (Black Friday) → every catalogue query is bounded (partners x page size) and has a statement timeout, so it fails fast with a 503 instead of queueing; imports and sending run on separate worker instances and slow down first (smaller batches), not the catalogue.

R5 - Being informed of new offers

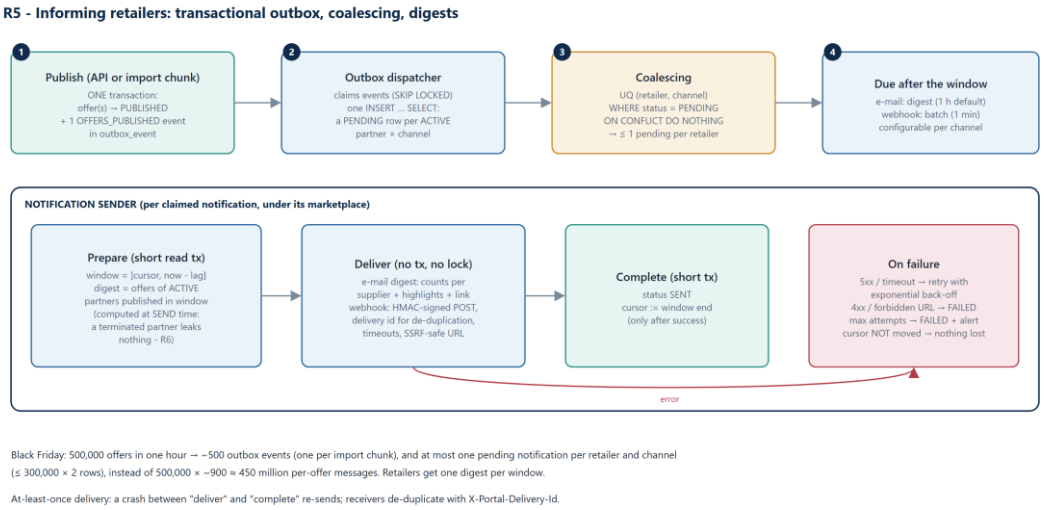


Figure 5 - Notifications

Transactional outbox: publishing (API or import chunk) writes the offers **and** one `OFFERS_PUBLISHED` event in the same transaction. **Coalescing:** one `INSERT ... SELECT` creates a PENDING notification per active partner and channel, with `ON CONFLICT DO NOTHING` on a partial unique index, so a retailer has **at most one** pending notification per channel. **Content at send time:** the digest lists only what the retailer can see now (BR-10), since its cursor. E-mail digests summarise per supplier with a link to the catalogue; large retailers get a **signed webhook** (HMAC-SHA256, delivery id, HTTPS only, SSRF blocked, no redirects, short timeouts). The cursor moves only after a 2xx.

> **When things go wrong**
>
> Receiver down → retries with back-off, 4xx fail at once; the cursor did not move, so the next digest carries the offers - nothing lost.
>
> Partnership terminated before sending → nothing leaks (computed at send time). Crash after delivery → re-sent once; receivers de-duplicate on the delivery id.
>
> Slow receivers → no delivery starts with under 30 s left on the 2-minute lease; the rest is released, so two workers never send the same notification.

Black Friday: 500,000 offers → ~500 outbox events → at most 300,000 x 2 notifications → one digest per retailer per window. Sending is the heaviest part: parallel sending (e.g. 20 per instance) on 4 worker instances sends 600,000 notifications in about 12 minutes; beyond that, a bulk e-mail provider and a broker fed by the same outbox.

R6 - End of a partnership

Termination sets `terminated_at` and cancels the subscriptions **in the same transaction**. Access disappears instantly because visibility is never stored: catalogue, subscribing and digests join only partnerships that are ACTIVE now. Termination locks the partnership `FOR UPDATE`, subscribing `FOR SHARE`, so no subscription slips through. A renewal waiting to start is terminated in the same transaction, so access never comes back on its start date. Data already pushed out cannot be taken back, so a `PARTNERSHIP_TERMINATED` webhook event (designed, not in the PoC) tells retailer systems to purge that supplier's offers, and the R9 extract lists terminations too. Being enforced at read time and by locks, R6 holds "whatever the way the platform is built".

> **When things go wrong**
>
> Termination during a running import of that supplier → the import continues (offers belong to the supplier, not to the partnership); the retailer is no longer an active partner, so it is not in the fan-out and its digest does not list them.

Termination at the same moment as a subscription → the two row locks serialise them: either the subscription is created first and then cancelled, or it is refused. Never a subscription that survives.

R7 - Security and compliance

- **Identity and access:** OIDC (Keycloak), MFA for administrators, OAuth2 client credentials for supplier systems; deny by default, role **and** ownership checks (404 for others' data). The PoC reads identity from headers; only one class changes.
- **Audit (ISO 27001), built:** append-only `audit_event` - who, what (before/after), when, correlation id - written in the same transaction as each change; the application role cannot UPDATE or DELETE it; production copies it to write-once storage. The PoC records the **organisation** (supplier, retailer or admin id); production records the **individual user** too (the token's `sub` claim), because ISO 27001 asks "which person", not only "which company".
- **GDPR. Minimisation:** only business contact data (company e-mail, website, address), no consumer data, never in logs. **Retention:** uploaded files 30 days, import reports 90 days, sent notifications 12 months, audit trail as long as the certification requires - each purged by a job with its own role. **Erasure versus an append-only audit:** the audit stores only ids and business values, never contact details, so erasing or anonymising the directory row makes the trail non-identifying without rewriting it; if personal values ever must be audited, they are encrypted with a per-person key that is deleted on erasure (crypto-shredding). **Extracts** exclude personal data by default. **Residency:** the dedicated cell, its backups and its keys stay in the customer's required region; the e-mail provider is a processor under a DPA.
- **Secrets:** webhook secrets encrypted with AES-256-GCM, key outside the database.
- **Application security:** parameterised SQL, DTOs at the boundary, request size limits, CSV-injection-safe exports, SSRF protection, least-privilege database roles, rate limits at the gateway, dependency scanning in CI.

R8 - Availability 99.9% and zero-downtime releases

Two or more stateless instances per cell across zones; managed PostgreSQL with failover; readiness and liveness probes. Rolling deployments with graceful shutdown (running work gets 25 s; leases resume anything interrupted). **Expand/contract** migrations keep each release compatible with the previous one. A new release goes to one cell first (canary).

When things go wrong

Database failover in the middle of a request → that transaction fails and rolls back; the pool reconnects to the new primary; reads are retried once, writes return 503 and clients retry - safe, because imports (file fingerprint), subscriptions and deliveries are idempotent.

Deploy while workers are running → running work gets 25 s to finish; anything cut off keeps its lease, which expires, and another instance resumes from the last committed chunk or notification. Nothing is lost or done twice.

R9 - Daily extract · R10 - Branding

R9: a scheduled job per marketplace reads the replica and writes the day's activity - offers, partnerships, subscriptions and notifications created or changed that day - as CSV/Parquet to object storage or SFTP. The **day is a Europe/Paris calendar day** `[00:00, next 00:00[`, so it lasts 23 or 25 hours on DST days. The extract is incremental (rows whose creation or change time falls in that day), idempotent per (marketplace, day), and comes with a manifest giving row counts and checksums per file; personal data is excluded by default. **R10:** a `marketplace_branding` record (logo, colour tokens); the Angular application resolves the marketplace from the sub-domain and applies PrimeNG theme tokens at start-up - no build per customer; colours are checked for contrast. The **digest e-mails** use the same record (logo, colours, sender name), since retailers see them as much as the portal.

When things go wrong

Extract fails or runs late → it is retried, and can be re-run for any past day because it is selected by time; the manifest is written last, so a reporting tool never loads half a day; an alert fires if a marketplace has no manifest by 06:00.

Missing logo or unreadable colours → colours that fail the WCAG contrast check are refused when saved; a missing or broken logo falls back to a neutral default theme, so the portal and e-mails always work.

5. Main technical decisions and alternatives

Decision	Alternatives	Why	Revisit when
Modular monolith, 3 layers	Microservices	One team, strong consistency (R6), no distributed transactions.	Independent teams or very different scaling per module.
Shared DB + RLS, plus cells	Schema or database per tenant for all; app filtering only	Isolation even against bugs, near-zero cost; silos only where a contract pays.	Many customers require separation → automated cells.
PostgreSQL as work queue	Kafka, RabbitMQ, SQS	Transactional with the data, one fewer system, runs locally.	Thousands of events/s sustained, or several consumers.
Digests computed at send time	One message per offer and retailer	450M messages are neither affordable nor wanted; enforces R6.	Retailers require per-offer real time.
One import pipeline, connectors at the edge	One pipeline per format	Same rules, report, idempotency and recovery for every format.	-
Status derived from dates	Stored status + nightly job	Always correct (BR-05), nothing to fail at midnight.	Reporting needs status history.
Rules guaranteed by constraints	Java only; SERIALIZABLE everywhere	Race-proof and cheap; no retry storms.	-
JDBC for bulk, JPA for aggregates	JPA everywhere; jOOQ	Per-entity overhead is wrong for 100k rows.	Complex queries → jOOQ.
Keyset pages, partial indexes, replica	OFFSET, Elasticsearch, cache	Constant cost per page; caches threaten R6.	Relevance search across marketplaces.

Risks and mitigations

- **RLS misconfigured** (application connecting as owner) → dedicated non-owner role; tests run as that role.
- **Queue tables grow and bloat** → partial indexes on waiting rows only, per-table autovacuum settings, monthly partitions dropped after retention.
- **Replica lag vs R6** → partnership checks on the primary. **Hostile webhooks** → timeouts, SSRF blocking, bounded retries, circuit breaker.

6. From today's application to the target

6.1 Migration path (each step shippable)

Step	What	Risk control
1. Tenant-ready schema	`marketplace_id` everywhere (expand), backfill, composite keys, NOT NULL (contract).	No downtime.
2. Isolation	Identity → marketplace pipeline, non-owner role, RLS policies.	Table by table; tests prove isolation.
3. Database guarantees	Unique, exclusion and check constraints after cleaning data.	Detection queries first.
4. Async foundations	Outbox and dispatcher; e-mail behind it; audit trail.	Feature flag, old path kept.
5. Imports	CSV, then JSON API, dry run; later Excel matrix and GDSN connectors.	Pilot with 2-3 suppliers.
6. Catalogue performance	Partial indexes, keyset pagination, replica, k6 in CI.	Measured before/after.

Step	What	Risk control
7. Operations, then marketplace #2, then the dedicated cell	Workers, probes, alerts, extracts, branding; automated cell creation.	Canary cell, rehearsed runbook.

Timeline (18 months): steps 1-3 in the first quarter; imports and notifications (4-5) by month 4, with marketplace #2 live; catalogue work (6) and the dedicated cell by month 6; the remaining marketplaces then join in waves of 3-4 per quarter, about 20 by month 12, leaving 6 months of margin before the growth target.

6.2 What I would deliberately not build yet

Not now	Because	Trigger
Microservices	Distributed failures we do not have.	Team topology, scaling needs.
Kafka / RabbitMQ	The PostgreSQL outbox handles our peak.	Thousands of events/s, several consumers.
Elasticsearch	~9,000 offers per retailer; pg_trgm suffices.	Relevance search, facets at scale.
Sharding, partitioning of offers	3M offers fit easily.	~50M rows; partition queue and audit tables first.
Application cache	Would show offers after termination (R6).	p95 over budget after indexes and replica; then per-supplier content only.

7. Proof of concept

Scope: R1, R3 and R5 as required, plus the rules they depend on (R6, BR-03 to BR-11) and the R7 audit trail. Java 21, Spring Boot 3.5, PostgreSQL 16, Flyway. No user interface (allowed). `docker compose up --build` starts PostgreSQL, the application and Mailpit (a local inbox showing the digests); the README walks through every behaviour with curl.

Area	In the PoC	In production
Identity	Headers X-Marketplace-Id / X-Actor-Role / X-Actor-Id	OIDC token; only `ActorContextInterceptor` changes
Isolation	Real: non-owner role, RLS, SECURITY DEFINER claims, composite keys	Plus dedicated cells
Imports	Real: CSV and JSON connectors, dry run, chunks, checkpoint, idempotency, report by code	Plus Excel/GDSN connectors, S3, retention
Notifications	Real: outbox, coalescing, digest, e-mail, signed webhooks, retries, leases	Plus parallel sending, preferences, circuit breaker
Audit, security	Real: append-only audit trail, encrypted secrets, size limits, SSRF rules	Plus write-once copy, gateway rate limits

Tests. 88 unit tests cover the domain rules and an **architecture test** that fails the build if a controller reaches a repository or SQL appears outside a repository. 66 integration tests run the whole application against a **real PostgreSQL** (Testcontainers) as the non-owner role, because RLS, the exclusion constraint and `ON CONFLICT` are exactly what is under test. Workers are called directly, so tests never sleep. Each bug found in review was first reproduced by a failing test.

Need	Tests (selection)
R1	`databaseHidesOtherMarketplaces_evenWithoutWhereClause`, `retailerOfMarketplaceA_cannotSeeOffersOfMarketplaceB`, `writingIntoAnotherMarketplace_isRejectedByTheDatabase`

Need	Tests (selection)
R3	`partialErrors_validLinesImported_invalidLinesReported`, `sameFileTwice_noDuplicates`, `sameRequestTwice_noDuplicates`, `dryRun_reportsWithoutChangingAnything`, `nulCharacterInCell_isALineError`
R5	`publication_andOutboxEvent_areAtomic`, `manyPublications_coalesceIntoOneDigest`, `partnershipTerminatedBeforeSending_nothingSent`, `webhook_isDeliveredAndSigned`, `slowDelivery_releasesTheRest_andNothingIsSentTwice`
R6, R7	`termination_cancelsSubscriptions_andRemovesAccess`, `concurrentOverlappingCreations_onlyOneSucceeds`, `offerChanges_areRecorded`, `trail_isAppendOnly`

Change-friendliness: a new offer field → `OfferTerms`, a migration, the column list; a new import format → one `OfferSourceAdapter`; a new notification channel → one `NotificationChannel`; a new error → one `ErrorCode`; the digest window → configuration only.

Known limitations. The full list, each with its fix, is in `EDGE_CASES_AND_DECISIONS.md` §9. The most important: identity comes from headers (OIDC in production); if the database ever refuses a line that Java accepted, the whole 1,000-line chunk fails and later lines are not imported (fix: replay that chunk line by line with savepoints); an import chunk that commits more than 10 s after its `published_at` could be skipped by a digest (fix: move the cursor only up to the oldest running transaction, `pg_snapshot_xmin`); offers published with a future start date are not announced on that day yet (a daily job); sending is sequential (parallel sending is designed); catalogue speed is designed but not yet measured with 3M offers.

Use of AI assistants

I used an AI coding assistant (Claude) as a pair programmer: to challenge the design, review the code for bugs, draft code, tests and documentation. Every decision, trade-off and line of code was reviewed and is owned by me; all tests run against a real PostgreSQL.